

axios

Séance 5 - Consommer des APIs externes proprement

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

fetch vs axios - GET/POST - params & headers - instances - intercepteurs - erreurs - Pokédex

Où on en est, où on va

Récap S4 et objectifs S5

Acquis (S4)

- Serveur Express : routes, middlewares
- CRUD complet avec Prisma (Neon)
- req.params, req.query, req.body
- Aperçu rapide d'axios (1 slide)
- Tester avec Postman / REST Client

Objectifs de la séance

- Comprendre ce qu'est un client HTTP
- axios vs fetch : pourquoi, quand
- GET / POST, params, headers, body
- Créer une instance (axios.create)
- Intercepteurs + gestion d'erreurs
- Brancher axios dans le projet Pokédex

L'esprit de la séance. Notre backend ne se contente plus de parler à SA base de données : il devient lui-même le CLIENT d'autres serveurs. On a déjà utilisé fetch pour aller chercher des Pokémon. Aujourd'hui on adopte axios, l'outil que la plupart des projets Node utilisent pour ça — plus court, plus sûr, plus configurable.

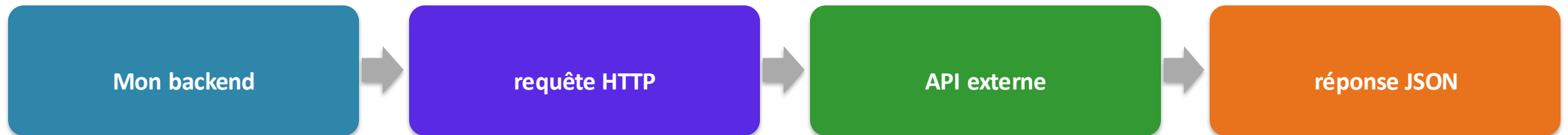
PARTIE 1

Pourquoi axios plutôt que fetch ?

Rappel : consommer une API

Notre backend devient client d'un autre serveur

Souvent, notre serveur a besoin de données qui viennent d'ailleurs : la météo, un taux de change, ou — comme dans notre projet — la liste des Pokémon. Pour les obtenir, il fait une requête HTTP vers une API externe, exactement comme un navigateur le ferait.



- Pour faire cette requête, il faut un client HTTP
- **Vu en S4** : fetch, natif depuis Node 18 (aucune installation)
- **Aujourd'hui** : axios, une librairie dédiée à installer
- Les deux font la même chose... mais pas avec le même confort

fetch fait le travail, mais...

Ce qu'il faut gérer soi-même avec fetch

La version fetch (vue avec le Pokédex) :

```
const reponse = await fetch(url);

// 1) fetch ne lève PAS d'erreur sur 404/500
if (!reponse.ok) {
  throw new Error('Pokemon introuvable');
}

// 2) il faut parser le JSON soi-même
const data = await reponse.json();

console.log(data.name);
```

TS

Les irritants de fetch

- Deux await (fetch + .json())
- Statut 404/500 = PAS une erreur -> vérifier reponse.ok à la main
- Pas de timeout intégré (il faut un AbortController)
- Headers / params plus verbeux

Rien de bloquant... mais sur un projet qui appelle souvent une API, ça se répète vite. axios supprime ces frictions.

La même chose avec axios

Plus court, et les erreurs HTTP deviennent des exceptions

La version axios :

```
import axios from 'axios';

// data est DEJA en JSON
const { data } = await axios.get(url);

console.log(data.name);
// un 404 declenche automatiquement le catch
```

TS

Ce qu'axios fait pour vous

- Parse le JSON tout seul (.data)
- Lève une erreur sur 4xx / 5xx
- timeout, baseUrl intégrés
- Syntaxe params/headers concise

En une ligne :

- **fetch** = bas niveau, natif, minimaliste
- **axios** = surcouche pratique, pensée pour les vrais projets
- **Même API au navigateur et côté Node** : on apprend une fois, on réutilise partout

PARTIE 2

Les bases d'axios

Installation et import

axios n'est pas natif : on l'ajoute au projet

1. Installer axios (dépendance de production) :

```
npm install axios
```

bash

2. L'importer dans un fichier TypeScript :

```
import axios from 'axios';
```

TS

À noter. Contrairement à fetch, axios ajoute une dépendance (et donc du poids). C'est le prix de ses fonctionnalités. axios fournit aussi ses propres types TypeScript : pas besoin d'un @types/axios séparé.

Vérifiez ensuite dans package.json que "axios" apparaît bien dans "dependencies".

Premier GET

axios.get renvoie une Promise -> on met await

```
import axios from 'axios';

async function getPokemon() {
  const reponse = await axios.get(
    'https://pokeapi.co/api/v2/pokemon/pikachu'
  );

  // les donnees sont DEJA en JSON
  console.log(reponse.data.name); // 'pikachu'
}
```

TS

Ce qui change vs fetch

- Pas de .json() à appeler
- Les données sont dans reponse.data
- Astuce : on déstructure `const { data } = await axios.get(url);`

Rappel async/await : axios.get(...) rend une Promise. On utilise await dans une fonction async (ou .then() si on préfère les callbacks).

Que contient la réponse axios ?

Un objet riche, pas seulement les données

```
const reponse = await axios.get(url);
```

TS

```
reponse.data      // le corps (deja en JSON)
reponse.status    // 200, 404, 500...
reponse.statusText // 'OK'
reponse.headers   // les en-tetes de reponse
reponse.config    // la config envoyee
```

À retenir

- 99 % du temps on veut .data
-
- status est utile pour logger
- ou décider d'un comportement
-
- headers : pagination, type de
- contenu, limites d'API...

Champ	Contenu	Équivalent fetch
reponse.data	le corps déjà parsé	await reponse.json()
reponse.status	code HTTP (nombre)	reponse.status
reponse.headers	en-têtes de réponse	reponse.headers

GET avec des paramètres (query string)

L'objet params plutôt que coller des ?cle=valeur

À éviter (concaténer à la main) :

```
await axios.get(`https://pokeapi.co/api/v2/pokemon?limit=${n}&offset=${o}`);
```

TS

Recommandé (objet params) :

```
const { data } = await axios.get(
  'https://pokeapi.co/api/v2/pokemon',
  { params: { limit: 20, offset: 0 } }
);
// URL finale : .../pokemon?limit=20&offset=0
```

TS

Avantage clé. axios encode les valeurs pour vous (espaces, accents, &, caractères spéciaux). Plus besoin de encodeURIComponent ni de bricoler la query string.

Envoyer des données : POST

Le 2e argument = le corps de la requête

```
const { data } = await axios.post(
  'http://localhost:3000/users',
  {                                // 2e arg = le corps
    email: 'alice@cmaisonneuve.qc.ca',
    name: 'Alice'
  }
);

console.log(data); // l'utilisateur cree
```

TS

axios fait le travail

- Sériialise l'objet en JSON
- automatiquement
-
- Ajoute le bon en-tête :
- Content-Type:
- application/json
-
- (avec fetch : JSON.stringify
- + header manuel)

Signatures à connaître : `axios.get(url, config)` / `axios.post(url, corps, config)` / `axios.put(url, corps, config)` / `axios.delete(url, config)`.

Headers, config et timeout

Le 3e objet de configuration

```
const { data } = await axios.get(url, {  
  headers: {  
    Authorization: 'Bearer MON_TOKEN',  
    Accept: 'application/json',  
  },  
  params: { limit: 20 },  
  timeout: 5000, // abandonne apres 5 secondes  
});
```

TS

À quoi ça sert

- headers : clés d'API, auth, format
- params : la query string
- timeout : éviter d'attendre l'infini
- baseUrl : préfixe commun (slide suivant)

vs fetch

- timeout intégré (fetch ne l'a pas)
- Objet config unique et lisible
- Pas de new Headers(...)
- Auth en une ligne

PARTIE 3

Aller plus loin : instances & erreurs

Créer une instance réutilisable

axios.create : ne plus répéter l'URL de base

```
// src/api/pokeapi.ts
import axios from 'axios';

export const pokeapi = axios.create({
  baseURL: 'https://pokeapi.co/api/v2',
  timeout: 5000,
});

// ailleurs : plus besoin de reecrire l'URL complete
const { data } = await pokeapi.get('/pokemon/pikachu');
```

TS

Bonne pratique. Une instance par service externe (une pour la PokéAPI, une pour une autre API...). On centralise baseURL, timeout et headers communs en un seul endroit — exactement l'idée du singleton `utils/prisma.ts`, mais pour les appels HTTP.

Les intercepteurs

Exécuter du code avant chaque requête / après chaque réponse

Un intercepteur est un middleware côté client : il s'exécute automatiquement pour TOUTES les requêtes d'une instance. Parfait pour ajouter un token ou logger.

```
// Avant chaque requete : ajouter le token automatiquement
pokeapi.interceptors.request.use((config) => {
  console.log('Appel ->', config.url);
  // config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Après chaque reponse : log du statut
pokeapi.interceptors.response.use((res) => {
  console.log('Recu <-', res.status);
  return res;
});
```

TS

Cas d'usage typiques : injecter une clé d'API partout, journaliser les appels, rafraîchir un token expiré, mesurer les temps de réponse. fetch n'a rien d'équivalent en natif.

Gérer les erreurs proprement

try/catch + isAxiosError + error.response

```
try {
  const { data } = await axios.get(url);
  return data;
} catch (e) {
  if (axios.isAxiosError(e) && e.response) {
    // le serveur a répondu (404, 500...)
    console.log(e.response.status);
    console.log(e.response.data);
  } else {
    // pas de réponse (réseau, timeout, DNS)
    console.log('Erreur réseau');
  }
  return null;
}
```

TS

Les 3 cas à distinguer

- axios LÈVE sur 4xx/5xx
- -> le catch se déclenche seul
-
- e.response existe :
- le serveur a répondu
- (.status, .data)
-
- PAS de e.response :
- problème réseau / timeout
-
- isAxiosError() : typage sûr

PARTIE 4

axios dans notre backend Pokédex

On refactorise recupererPokemon

La fonction du Pokédex, version axios

TS

```
import axios from 'axios';
const POKEAPI = 'https://pokeapi.co/api/v2';

async function recupererPokemon(nom: string) {
  try {
    const { data } = await axios.get(`${POKEAPI}/pokemon/${nom}`);
    const hp = data.stats.find((s: any) => s.stat.name === 'hp')?.base_stat ?? 0;
    return {
      numeroPokedex: data.id, nom: data.name, pointsVie: hp,
      taille: data.height / 10, poids: data.weight / 10,
      imageUrl: data.sprites.front_default,
    };
  } catch (e) {
    return null; // 404 ou reseau -> pokemon introuvable
  }
}
```

Ce qu'on a gagné

Avant (fetch) vs après (axios) dans recupererPokemon

Avant - fetch :

```
const rep = await fetch(
  `${POKEAPI}/pokemon/${nom}`
);
if (!rep.ok) return null; // a la main
const data = await rep.json();
// ... suite
```

TS

Après - axios :

```
try {
  const { data } = await axios.get(
    `${POKEAPI}/pokemon/${nom}`
  );
  // ... suite
} catch { return null; }
```

TS

- Un seul **await** au lieu de deux (plus de `.json()`)
- Plus de **if (!rep.ok)** : le catch gère le 404 naturellement
- Le 'introuvable' et le 'réseau' passent par le même catch -> return null

La route qui combine axios + Prisma

POST /pokedex/capturer/:nom -> API externe puis base Neon

```
import prisma from '../utils/prisma';

routerPokedex.post('/capturer/:nom', async (req, res) => {
  // 1) axios : recuperer depuis la PokeAPI
  const donnees = await recupererPokemon(req.params.nom);
  if (!donnees) {
    return res.status(404).json({ erreur: "Pokemon introuvable" });
  }
  // 2) Prisma : enregistrer dans Neon
  try {
    const pokemon = await prisma.pokemon.create({ data: donnees as any });
    res.status(201).json({ message: `${pokemon.nom} capture !`, pokemon });
  } catch (e) {
    res.status(400).json({ erreur: 'Deja capture ?' });
  }
});
```

TS

PARTIE 5

Récap, choix et pratique

fetch vs axios - récapitulatif

Le tableau à retenir

Critère	fetch	axios
Installation	natif (Node 18+)	npm install axios
Parsing JSON	await reponse.json()	auto (reponse.data)
Erreur sur 404/500	non (vérifier .ok)	oui (try/catch)
Query params	à concaténer / encoder	objet params
Corps POST	JSON.stringify + header	objet (auto)
Timeout	AbortController	intégré (timeout)
baseUrl / instance	non	axios.create
Intercepteurs	non	oui

Alors, fetch ou axios ?

Aucun n'est « meilleur » dans l'absolu

fetch suffit si...

- Quelques requêtes simples
- Pas envie d'ajouter une dépendance
- Script rapide / projet léger
- Tu maîtrises `response.ok`
- Front moderne sans besoins avancés

axios brille si...

- Beaucoup d'appels vers la même API
- Besoin de `baseUrl`, headers, timeout
- Gestion d'erreurs propre (`try/catch`)
- Intercepteurs (auth, logs)
- Code plus court et lisible

Pour notre projet (backend qui appelle souvent la PokéAPI) : axios est le bon choix.

À vous de jouer

Refactoriser le Pokédex avec axios

- 1. Installer **axios** dans le projet Pokédex (npm install axios)
- 2. Remplacer **fetch** par **axios.get** dans `recupererPokemon`
- 3. Déstructurer la **réponse** : `const { data } = await axios.get(...)`
- 4. Supprimer le **if (!reponse.ok)** -> le `try/catch` suffit
- 5. Créer une instance **pokeapi** avec `axios.create + baseUrl`
- 6. **Bonus** : ajouter un `timeout` de 5 s et gérer `e.response.status`

Objectif : même comportement qu'avant, mais avec un code plus court et plus robuste.

Récapitulatif de la séance

Ce qu'il faut retenir d'axios

Les bases

- npm install axios
- axios.get(url)
- données dans .data
- params pour la query
- post(url, corps)

Plus loin

- axios.create({ baseURL })
- timeout intégré
- headers / auth
- intercepteurs
- (requête + réponse)

Erreurs & projet

- try/catch naturel
- isAxiosError(e)
- e.response.status / .data
- Pokédex : axios + Prisma
- capturer -> Neon

La commande et l'import à mémoriser :

```
npm install axios
```

```
import axios from 'axios';  
const { data } = await axios.get(url, { params, headers, timeout });
```

TS

Pour la prochaine séance

On consolide et on prépare la suite

À faire de votre côté

- Refactoriser le Pokédex avec axios
- Créer une instance pokeapi (baseURL)
- Gérer les erreurs avec isAxiosError
- Tester capturer/:nom dans Postman
- Vérifier la persistance dans Neon
- Pousser le tout sur GitHub

Au programme ensuite

- Validation des entrées (Zod)
- Pagination & filtres via params
- Authentification (tokens, en-têtes)
- Intercepteurs avancés (refresh token)
- Puis : React (frontend) qui appelle
- notre API avec axios

Votre backend sait maintenant consommer n'importe quelle API externe proprement.

Et bientôt, c'est votre frontend React qui utilisera axios pour parler à CE backend.



Questions ?

Prochaine étape : refactoriser le Pokédex avec axios.

Installez, remplacez fetch, testez. Et attrapez-les tous.